Optimal Precision Formats for G4 GPU

https://gemini.google.com/app/f9d742ef785a37c4

User prompt: What is the best precision format for the G4 GPU on Google Colab(RTX 6000 PRO server edition)?I have access to it on the Colab plan.

Response: The optimal precision format for the **NVIDIA RTX PRO 6000 (Blackwell Server Edition)** on Google Cloud G4 runtimes depends on whether you are running inference, fine-tuning, or general tensor computations. Because Blackwell features 5th-generation Tensor Cores and 2nd-generation Transformer Engine, hardware support extends from standard 16-bit down to native sub-byte floating-point formats.

Recommended Precisions by Workload

Workload	Recommended Precision	Why It's the Best Choice
LLM Inference (Max Speed / Serving)	**FP4 (NVFP4)** or **FP8 (E4M3)**	Blackwell has native hardware acceleration for FP4 and FP8 Tensor Core GEMMs. FP4 roughly doubles throughput over FP8 while slashing VRAM footprint, while FP8 retains full baseline quality with virtually zero perplexity loss.
Model Training & Fine-Tuning (Default)	**BF16 (Bfloat16)**	Shares the 8-bit dynamic range of FP32. It prevents underflow/overflow without needing dynamic loss scaling (making it strictly preferred over FP16 on modern architectures).
High-Throughput Training (Optimized)	**FP8 Mixed Precision**	Forward and backward passes execute in FP8 (E4M3 for weights/activations, E5M2 for gradients) via NVIDIA Transformer Engine, doubling GEMM math throughput over BF16 while preserving FP32 master weights.
Standard FP32 PyTorch Operations	**TF32 (TensorFloat-32)**	Drop-in acceleration for FP32 matrix multiplications. Uses Tensor Cores to deliver near-FP16 speed with FP32 dynamic range without altering your loss or convergence.

Detailed Breakdown

1. Inference: FP8 (E4M3) and FP4 (NVFP4)

- **FP8 (E4M3):** If you are serving large models (such as Llama, Mistral, or Qwen) with frameworks like **vLLM**, **SGLang**, or **TensorRT-LLM**, run the model with FP8 weight and KV-cache quantization. It yields near-FP16 accuracy with roughly half the VRAM demand and up to 2x the token generation rate.
- **FP4 (NVFP4):** Unlike older Ada Lovelace cards (which emulated FP4 in software), Blackwell's 5th-gen Tensor Cores include native FP4 arithmetic units. When running heavily constrained memory workloads or serving massive context windows, quantized FP4 checkpoints provide the highest possible tokens/sec.

2. Fine-Tuning & Training: BF16 vs. FP16

- **Use BF16 instead of FP16:** FP16 has only a 5-bit exponent and 10-bit mantissa, making it prone to exploding or vanishing gradients during backpropagation unless manually wrapped in `torch.cuda.amp.GradScaler`. BF16 matches FP32's 8-bit dynamic range, ensuring training stability with identical speed and memory usage.
- In Hugging Face or PyTorch:

```
import torch

# Model loading
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto"
)

# TrainingArguments
training_args = TrainingArguments(
    bf16=True,
    fp16=False,
    ...
)
```

3. Standard Matrix Multiplications: TF32

- By default, PyTorch may evaluate raw `float32` tensors on regular CUDA cores instead of Tensor Cores. Always enable TF32 at the top of your Colab notebook to allow Tensor Cores to accelerate standard FP32 matmuls:

```
import torch
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True
```

For the 96 GB GDDR7 VRAM pool on the G4 runtime, **BF16** is the most reliable and plug-and-play format for LoRA/full fine-tuning, while **FP8** (via `bitsandbytes` or `vLLM`) is the gold standard for maximum throughput inference. Avoid legacy **FP16** unless a specific library or legacy checkpoint explicitly lacks BF16 support.

Sources

1. Accelerating Computer Vision with Mixed Precision - NVlabs

---

User prompt: Yes, but then again, what is the precision format I should use for maximum token generation? These are the standard precision formats you're talking about. What will maximize token generation?

Response: For absolute maximum token generation (peak tokens per second), the format to use on Blackwell is **NVFP4 (4-bit Floating Point with Microscaling)**, paired with an **FP8 or FP4 KV cache**.

If your sole goal is raw generation speed and maximum serving throughput, NVFP4 beats every standard format (FP8, INT8, BF16) on this hardware.

Why NVFP4 Maximizes Token Generation

Token generation occurs in two distinct hardware regimes, and NVFP4 dominates both on the Blackwell architecture:

1. Low Batch / Interactive Decode (Memory-Bandwidth Bound)

Generating tokens one by one for an individual user or small batch sizes is strictly bottlenecked by the GPU's GDDR7 memory bandwidth—the GPU has to stream the entire model's weights from memory into the compute units for every single token produced.

- **BF16:** Transfers **2 bytes** per parameter per token.
- **FP8:** Transfers **1 byte** per parameter per token (~2× decode speedup).
- **NVFP4:** Transfers **0.5 bytes** per parameter per token (**~3.5× to 4× decode speedup** over BF16).

By reducing weight data transfer to 4 bits per weight, the memory bus feeds the compute units at maximum speed, maximizing single-stream and small-batch token output.

2. High Batch / Concurrent Serving (Compute Bound)

When batch sizes scale up or during long prompt evaluation (prefill), the bottleneck shifts to the Tensor Cores.

- Blackwell's 5th-generation Tensor Cores have native hardware units dedicated to FP4 matrix math.
- NVFP4 delivers **double the raw TFLOPS of FP8** and **quadruple the TFLOPS of BF16**.
- Unlike legacy INT4 (which often required runtime dequantization overhead on older architectures), Blackwell executes native NVFP4 GEMMs directly inside the Tensor Cores.

Token Generation Throughput Comparison

Precision Configuration	Relative Token Generation Speed	Quality Retention	Memory Footprint (Weights)
NVFP4 (W4A4 + FP8/FP4 KV)	**100% (Maximum Speed)**	97–99% of baseline (via microscaling)	~0.5–0.6 bytes/param
FP8 (W8A8 + FP8 KV)	**~50–60% of NVFP4**	>99.5% (virtually lossless)	1 byte/param
AWQ / GPTQ (INT4-Weight, FP16-Activation)	**~40–50% of NVFP4**	95–98%	~0.6 bytes/param (dequant overhead)
BF16 / FP16 Baseline	**~20–25% of NVFP4**	100% (Reference)	2 bytes/param

The Decisive Factor: The KV Cache

Quantizing model weights to 4-bit is only half the battle. If your Key-Value (KV) cache remains in 16-bit, long generation sequences and deep context windows will saturate the memory bandwidth and stall token output.

To achieve maximum tokens/second throughout long sequences:

1. **Weights & Activations:** NVFP4 (`fp4` / `nvfp4` quantization schemes via TensorRT-LLM or vLLM).
2. **KV Cache:** Set to **FP8 (`fp8_e4m3`)** or **FP4**.

In inference engines like **vLLM** or **SGLang**:

```
# Example serving flag combination for maximum decode speed
vllm serve <model-nvfp4-checkpoint> \
    --quantization fp4 \
    --kv-cache-dtype fp8 \
    --gpu-memory-utilization 0.95
```

If your software pipeline does not yet provide an NVFP4 checkpoint for your specific model architecture, the next fastest option that runs at native hardware speed on the RTX PRO 6000 is **FP8 (W8A8 with FP8 KV Cache)**.
